



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina
Análise Numérica**

Relatório de implementações realizadas

por Reinan Lopes Argolo

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilhéus – BA

2026

Sumário

1	Linguagem Escolhida e Justificativas	4
2	Método da Bissecção	5
2.1	Estratégia de Implementação	5
2.2	Estrutura dos Arquivos de Entrada/Saída	6
2.3	Problemas Teste	7
2.3.1	Problema Teste 1: Tempo de Subida de um Amplificador	7
2.3.2	Problema Teste 2: [Nome do Problema 2]	9
2.3.3	Problema Teste 3: [Nome do Problema 3]	9
2.4	Dificuldades Enfrentadas	9
3	Método 2	10
3.1	Estratégia de Implementação	10
3.2	Estrutura dos Arquivos de Entrada/Saída	10
3.3	Problema teste 1, 2, 3...	10
3.4	Dificuldades enfrentadas	10
4	Considerações Finais	11

Lista de Figuras

1	Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$	8
---	--	---

Lista de Tabelas

1	Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$	8
---	---	---

1 Linguagem Escolhida e Justificativas

Para o desenvolvimento dos algoritmos e a execução das rotinas computacionais propostas neste relatório de métodos numéricos, optou-se pela utilização da linguagem de programação Python. A escolha dessa ferramenta justifica-se, pela sua expressiva adoção na comunidade científica e pela vasta disponibilidade de bibliotecas para manipulação matemática e computação científica [3].

Entre as principais extensões que fundamentam essa decisão, destaca-se a biblioteca *NumPy*, a qual fornece suporte nativo e eficiente para o processamento de arranjos e matrizes multidimensionais. Essa biblioteca implementa operações de álgebra linear com rotinas de baixo nível, garantindo o alto desempenho exigido nos cálculos numéricos iterativos [1]. Em complemento, o uso do ecossistema *SciPy* oferece um conjunto de algoritmos fundamentais para interpolação, integração numérica, otimização e resolução de sistemas lineares e não lineares [4].

Ademais, Python caracteriza-se por uma sintaxe de alto nível e semântica clara, o que favorece a legibilidade do código e permite a prototipagem rápida de abstrações matemáticas [2]. Dessa forma, a redução da complexidade de implementação viabiliza um foco maior na análise de convergência, na estabilidade dos métodos e na mitigação de erros de truncamento e arredondamento, requisitos indispensáveis para a consistência deste trabalho acadêmico.

2 Método da Bissecção

O Método da Bissecção é uma abordagem numérica clássica e incondicionalmente convergente para a determinação de raízes reais de equações não lineares. Fundamentado no Teorema de Bolzano, o método opera mediante a divisão sucessiva de um intervalo fechado $[a, b]$, no qual a função contínua $f(x)$ apresenta sinais opostos em seus extremos ($f(a) \cdot f(b) < 0$). Embora possua uma taxa de convergência linear — exigindo um número razoável de iterações —, sua robustez o torna ideal para isolamento inicial de raízes ou resolução de problemas onde a estabilidade é prioritária em relação à velocidade computacional.

2.1 Estratégia de Implementação

A concepção computacional do algoritmo foi estruturada na linguagem Python, priorizando a estabilidade numérica e a modularidade do código. A implementação principal encontra-se encapsulada no módulo `src/metodos/zeros_de_funcao.py`.

Código 1: Código de implementação do Método da Bissecção

```
1 def bisseccao(funcao, a, b, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(max_iter):
5         c = (a + b)/2
6         fc = funcao(c)
7         denominador = a + b
8
9         if denominador != 0:
10             er = abs((a - b) / denominador)
11         else:
12             er = abs(b - a)
13
14         historico.append({
15             'iter': i, 'a': a, 'b': b, 'c': c,
16             'f(c)': fc, 'erro': er
17         })
18
19         if fc == 0 or er < tol:
20             return c, i, historico
21
22         if utils.teorema_bolzano(funcao, a, c):
23             b = c
```

```

24         else:
25             a = c
26
27     return c, max_iter, historico

```

Para a formulação matemática do erro iterativo e visando superar limitações do ambiente de programação, estabeleceu-se uma métrica adaptativa. Originalmente, o erro relativo ponderado pela amplitude do intervalo é definido como $\epsilon_{rel} = \left| \frac{a_k - b_k}{a_k + b_k} \right|$. Contudo, constatou-se que intervalos simétricos em relação à origem ($a_k = -b_k$) induziriam o interpretador Python a lançar uma exceção de divisão por zero (`ZeroDivisionError`). Para mitigar este risco, implementou-se um fluxo condicional: caso o denominador seja nulo, a métrica transita automaticamente para o erro absoluto $\epsilon_{abs} = |b_k - a_k|$.

Os critérios de parada foram definidos para garantir o término do laço em tempo finito, compreendendo:

1. **Raiz exata:** Interrupção imediata caso a avaliação funcional no ponto médio resulte exatamente em zero ($f(c_k) = 0$), limitando processamento desnecessário por flutuação de ponto flutuante.
2. **Tolerância (`tol`):** O método encerra com sucesso caso a métrica de erro calculada seja estritamente menor que a tolerância estipulada.
3. **Esforço computacional (`max_iter`):** Uma trava de segurança máxima de iterações para abortar o processo caso a convergência não seja alcançada no limite imposto.

2.2 Estrutura dos Arquivos de Entrada/Saída

Visando aderência aos princípios de responsabilidade única e reprodutibilidade científica, a persistência de dados foi dissociada da lógica matemática. Os diretórios foram organizados hierarquicamente:

- **Dados de Entrada (`dados/entradas/`):** Estruturados em arquivos de texto simples (`.txt`). Foram escolhidos por permitirem fácil edição manual dos parâmetros de controle (limites a e b , tolerância e limite de iterações) sem necessidade de reconfiguração no código-fonte.
- **Dados de Saída (`dados/saida/<questao>/`):** Os resultados iterativos foram consolidados em arquivos de valores separados por vírgula (`.csv`). A escolha do formato CSV justifica-se por sua interoperabilidade universal, permitindo a ingestão direta por bibliotecas de análise de dados (como Pandas) e por pacotes

de renderização gráfica em LaTeX (como TikZ/pgfplots), viabilizando análises quantitativas automatizadas e plots de convergência nativos no documento final.

2.3 Problemas Teste

A verificação e validação da implementação computacional foram conduzidas por meio de problemas aplicados extraídos da literatura de referência de Cálculo Numérico (Franco, Neide). O objetivo desta seção é atestar o funcionamento, a eficiência e a validade matemática das soluções encontradas.

2.3.1 Problema Teste 1: Tempo de Subida de um Amplificador

O primeiro problema propõe a determinação do tempo de subida de um amplificador eletrônico, compreendido como o intervalo temporal necessário para que a resposta $g(T)$ avance de 10% a 90% de seu valor assintótico, sendo a função normalizada expressa por:

$$g(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} \quad (1)$$

O cálculo do tempo de subida, $\Delta T = T_{0.9} - T_{0.1}$, exige a determinação das raízes das funções auxiliares:

$$f_1(T) = 0.9 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (2)$$

$$f_2(T) = 0.1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (3)$$

O algoritmo processou $f_1(T)$ com intervalo inicial $[1.0, 2.0]$ e tolerância 10^{-5} . A Tabela 1 exibe a progressão numérica.

A convergência foi atingida na 16ª iteração, com o erro caindo abaixo da tolerância ($\epsilon_{16} \approx 6.92 \times 10^{-6}$), retornando $T_{0.1} \approx 1.102058$. A Figura 1 ilustra este amortecimento da magnitude funcional, que demonstra a redução progressiva assintótica característica da contração de intervalo.

Tabela 1: Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$.

Iteração	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.000000	2.000000	1.500000	0.09115317	0.33333333
1	1.000000	1.500000	1.250000	0.03153233	0.20000000
2	1.000000	1.250000	1.125000	0.00466937	0.11111111
3	1.000000	1.125000	1.062500	-0.00785071	0.05882353
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.102051	1.102112	1.102081	0.00000322	0.00002769
15	1.102051	1.102081	1.102066	0.00000014	0.00001385
16	1.102051	1.102066	1.102058	-0.00000140	0.00000692

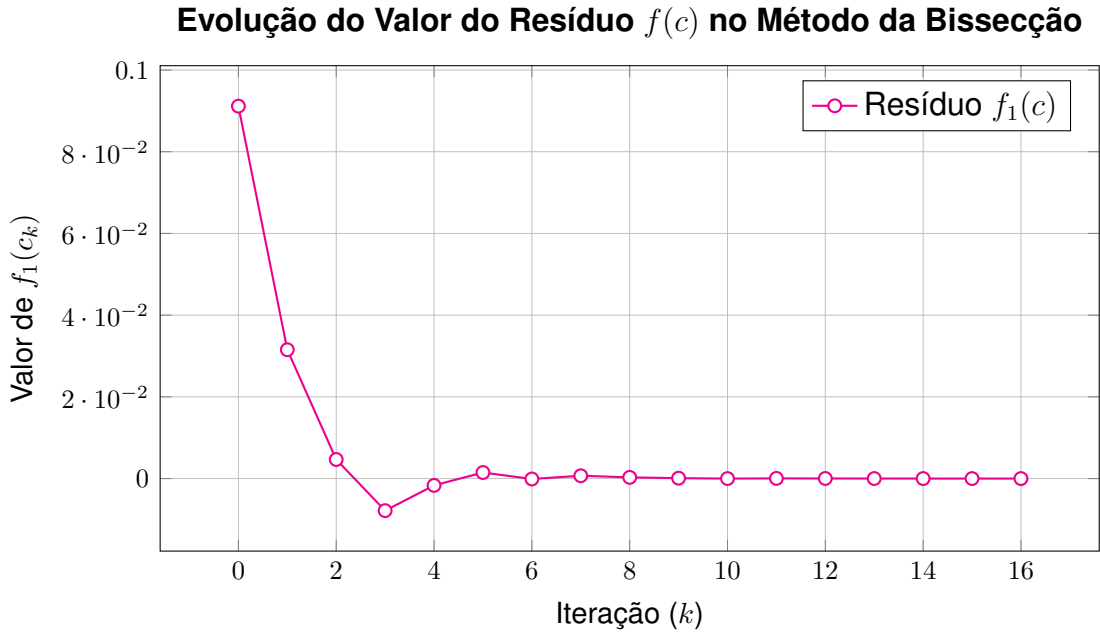


Figura 1: Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$.

Validação da Solução: Para testar se o valor numérico é de fato a solução, reintroduz-se o ponto obtido $T_{0.1} = 1.102058$ na equação original:

$$f_1(1.102058) = 0.9 - \left(1 + 1.102058 + \frac{(1.102058)^2}{2} \right) e^{-1.102058} \approx -0.00000140 \quad (4)$$

Observa-se que o resíduo apresenta magnitude estritamente compatível com a precisão exigida, confirmando tratar-se da raiz buscada. Procedimento análogo para $f_2(T)$ rendeu $T_{0.9} \approx 5.322320$. Consequentemente, o tempo de subida validado pelo método é $\Delta T = 5.322320 - 1.102058 = 4.220262$. Os resultados demonstram que a implementação comporta-se com a eficiência teórica esperada $\mathcal{O}(\log(1/\epsilon))$.

2.3.2 Problema Teste 2: [Nome do Problema 2]

2.3.3 Problema Teste 3: [Nome do Problema 3]

2.4 Dificuldades Enfrentadas

Durante o ciclo de desenvolvimento, duas barreiras fundamentais requisitaram análise e readequação de código:

A primeira dificuldade decorreu de uma limitação algorítmica associada à manipulação em linguagem Python envolvendo o cálculo de erro relativo. O equacionamento inicial do erro demandava a divisão pela soma dos limites do intervalo $(a_k + b_k)$. Em problemas cujo domínio submetido contemplasse raízes simétricas (como, por exemplo, funções ímpares centradas na origem onde $a_k = -b_k$), o denominador se tornava nulo, causando o encerramento abrupto da aplicação mediante a exceção `ZeroDivisionError`. A transição adaptativa do erro relativo para o erro absoluto nesses pontos críticos exigiu criteriosa instrumentação de *if-statements*, solucionando a limitação do ambiente sem desprezar o rigor numérico.

A segunda dificuldade envolveu a aderência aos padrões de projeto de software estruturado. A intenção de isolar a lógica matemática da escrita de arquivos resultou na necessidade de encapsular o estado iterativo. Em iterações profundas, manter um vetor histórico de dicionários em memória, para apenas no final transferi-los para a rotina de escrita CSV (camada de controle), demonstrou inicialmente um ligeiro sobrepeso na alocação de variáveis. A superação exigiu refinamento na forma como o Python instancia e mapeia essas estruturas temporárias. Ainda que houvesse maior complexidade inicial no planejamento desta abstração, as hipóteses projetuais confirmaram-se válidas, viabilizando o uso de dados de saída padronizados independentemente do método matemático em execução.

3 Método 2

3.1 Estratégia de Implementação

Descreva as escolhas que teve que fazer para poder superar eventuais dificuldades com limitações da linguagem, do ambiente de programação, etc. De que forma resolveu tratar os erros, critérios de parada, formato de saída e quaisquer outros fatores importantes para a solução dos problemas.

3.2 Estrutura dos Arquivos de Entrada/Saída

Descreva o formato dos arquivos de entrada e saída e justifique as escolhas.

3.3 Problema teste 1, 2, 3...

Use problemas das seções “Problemas Aplicados a Projetos” (ao final de cada capítulo) do livro Cálculo Numérico, de Neide Franco, para demonstrar que a sua implementação funciona e traz resultados da forma mais eficiente que lhe foi possível. Use, ao menos 3 problemas. Mostre os resultados e teste se eles realmente são a solução. Se não funcionar e você não entender o porquê, explique o que tentou e faça hipóteses.

3.4 Dificuldades enfrentadas

(Descreva as dificuldades aqui)

4 Considerações Finais

(Escreva suas considerações finais aqui)

Referências

- [1] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [2] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib*. Apress, 2 edition, 2018.
- [3] Travis E Oliphant. Python for scientific computing. *Computing in Science & Engineering*, 9(3):10–20, 2007.
- [4] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.